

# Introduction

I personally find that the ability to maintain focus during independent study sessions is a very common challenge faced by students especially in environments in this day and age where digital distractions are constantly present. Devices such as smartphones are significantly reducing the productivity and negatively impacting learning for not just me but most students around the world. While there are tools available such as timers or screen trackers that attempt to address this issue, they do often rely on manual input or they simply fail to identify the cause of your distraction.

To also address this problem the project i made is an ai powered study focus tracker designed to monitor users behaviour in real time using computer vision. The system i built uses your webcam to analyse your facial position, gaze direction and the presence of a smartphone being used in order to determine weather a user is focused, distracted or away. This project is primarily aimed at students engaging in independent study who want an automated way to understand and improve their concentration.

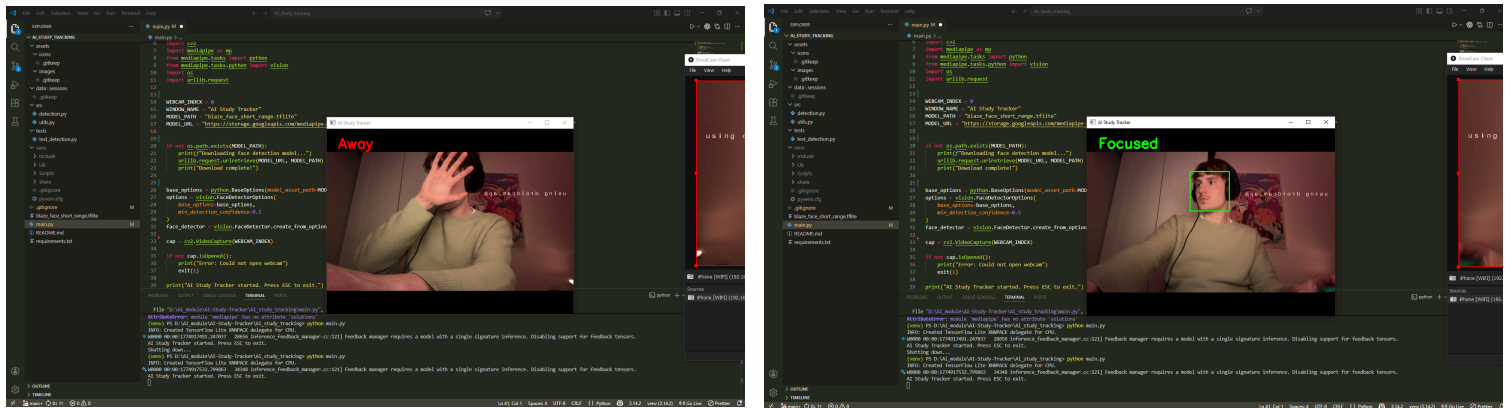
Using Artificial intelligence is a necessary component for this solution because detecting a user's attention from visual input like a webcam requires you to interpret complex facial landmarks/geometry and behavioural patterns in real time.

## Why Ai Was Necessary

The traditional ways of tracking focus during your study sessions are usually quite limited in what they can actually measure. Tools that are used such as timers, screen trackers or even manual logging rely on the user to input information and don't really give users much insight into what is actually causing a user's distraction. For example, a timer could show how long someone planned to study for however it can't tell whether they were actually studying the whole time (being focused or distracted during that time).

One non AI solution i could think of to this problem would be to track keyboard or mouse activity or even monitoring which applications are open. However, these approaches don't really reflect real life behaviour. Someone could be staring at their screen reading without interacting with their machine or using their phone whilst still technically being active on their computer. These methods can't really track where a user is looking which I think is a key part of understanding a users actual focus.

Artificial intelligence in my system allows the system to analyse your visual input from a webcam and then make sense of it in real time. By using computer vision the application can detect lots of different facial features, track head positioning and also estimate where the user is looking. This makes it possible to decide weather the user is focused on their work, distracted looking away or just not present at all.



*(real time simple focus detection using camera input)*

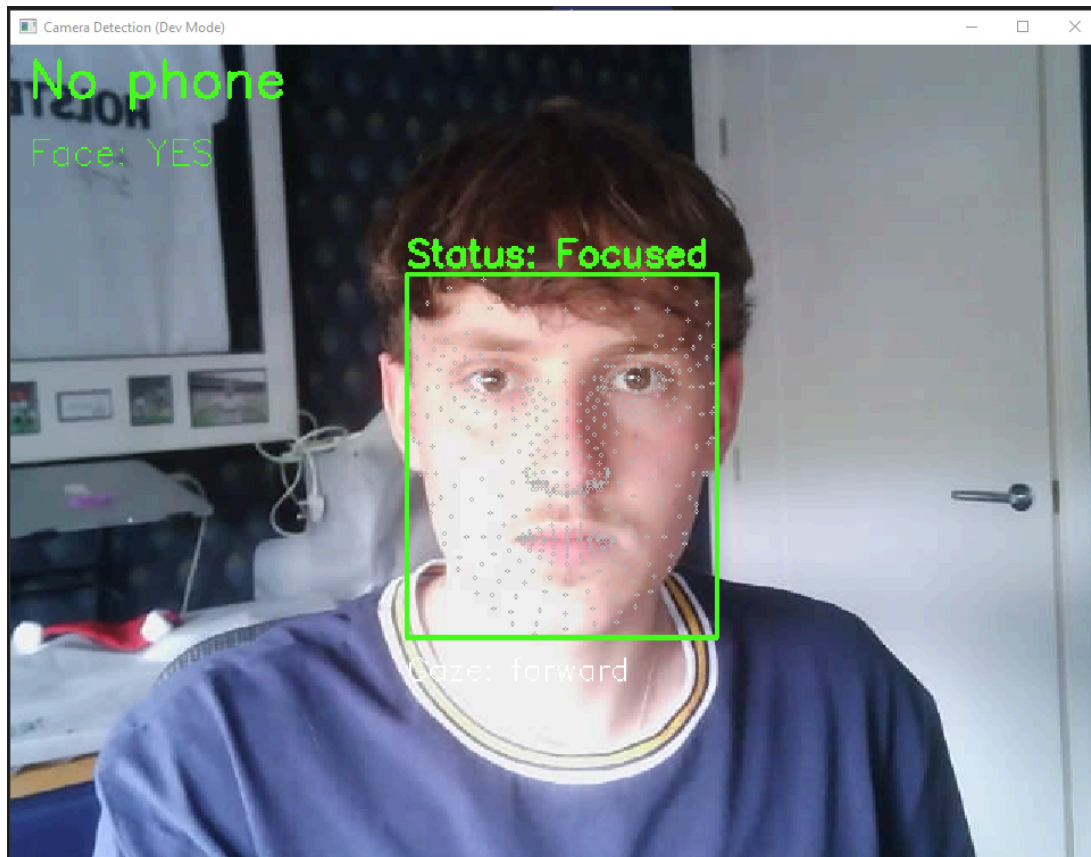
The addition i made later down the line of object detection improves this further by identifying specific distractions such as the main one of using a mobile phone. This is important because gaze direction use on its own can be unclear and not fully detect this issue whilst object detection has its own limits. For example looking down could mean things such as reading notes, looking at keyboard/mouse or checking a mobile phone. By putting both gaze tracking and object detection together the system can make a way more accurate decision about the users behaviour.

Overall in this project Artificial intelligence is not just a useful added feature however it is a necessary part of how the system works. Interpreting visual behaviour in real time is not something that can just be done reliably with traditional programming alone which is why AI is most definitely essential for this type of project I created.

## Choice of Ai Tools

To build the system I needed which was capable of analysing a user's focus in real time it was really necessary to use existing computer machine learning libraries rather than developing and training my own models from scratch. The project primarily uses [Mediapipe](#) and [OpenCV](#) also with a pretrained object detection model for identifying mobile phones.

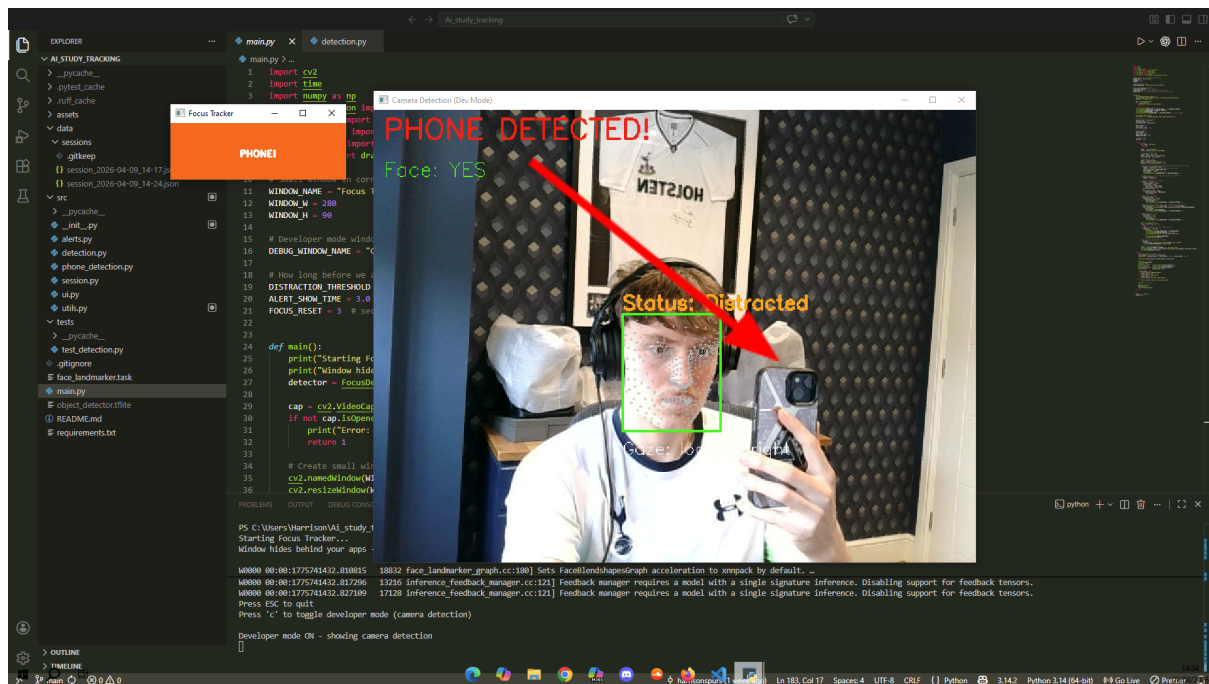
Googles [Mediapipe](#) LLM was chosen basically as the core Ai framework as it provides highly optimised, pre trained models for real time face and object detection. In particular the face landmark model was used to detect and detail important facial landmarks including key points around the eyes, nose and head. These are the landmarks that make it possible to estimate a user's head position and gaze direction with a good level of accuracy. One of the main advantages of [Mediapipe](#) is that it runs efficiently on a standard CPU which means the application doesn't require any high end hardware to run in real time. For example my home laptop is extremely old and can barely run basic tasks however runs this application perfectly even with a low standard built in webcam.



*(Facial landmark detection used to estimate head position and gaze direction)*

[OpenCV](#) was used in this project alongside [Mediapipe](#) to handle the video processing side of the system. This basically includes capturing frames from the webcam, converting image formats when its necessary in the system and drawing visuals such as overlays and status indicators over webcam footage. [OpenCV](#) acts as the main pipeline that feeds all of the image data into the ai model so that we can display the results back to the user which definitely makes it an essential part of my system.

In addition to the face tracking the project uses an object detection model called "[EfficientDet Lite](#)" to identify if a mobile phone was in the camera frame prompting the user to put it down and get back to studying. This was an important design decision because i found that gaze tracking alone is not always reliable when trying to identify distraction. For example a user could be looking down reading notes or simply glancing at their keyboard to see what they are typing however with gaze tracking this could be seen as a user looking down at their phone without actual object detection in place. By adding object detection my system was able to classify situations where a phone is present way more confidently improving my overall accuracy of this application.



*(phone detection using object detection)*

These pre trained models were used instead of training a custom model from scratch. Training a reliable model would require a large dataset of faces in different attention states as well as large computer resources. In contrast, [Mediapipe's](#) models have already been trained on large scale datasets and are extremely well optimised for real time use which was perfect for my situation. Using these models allowed the project to focus on interpreting the data they provide rather than trying to recreate a complex machine learning system.

Overall, the combination of [Mediapipe](#), [OpenCV](#) and the pre trained object detection model ([EfficientDet Lite](#)) provided the project a good balance between performance, accuracy and my overall development efficiency which makes them the perfect appropriate choices for this project.

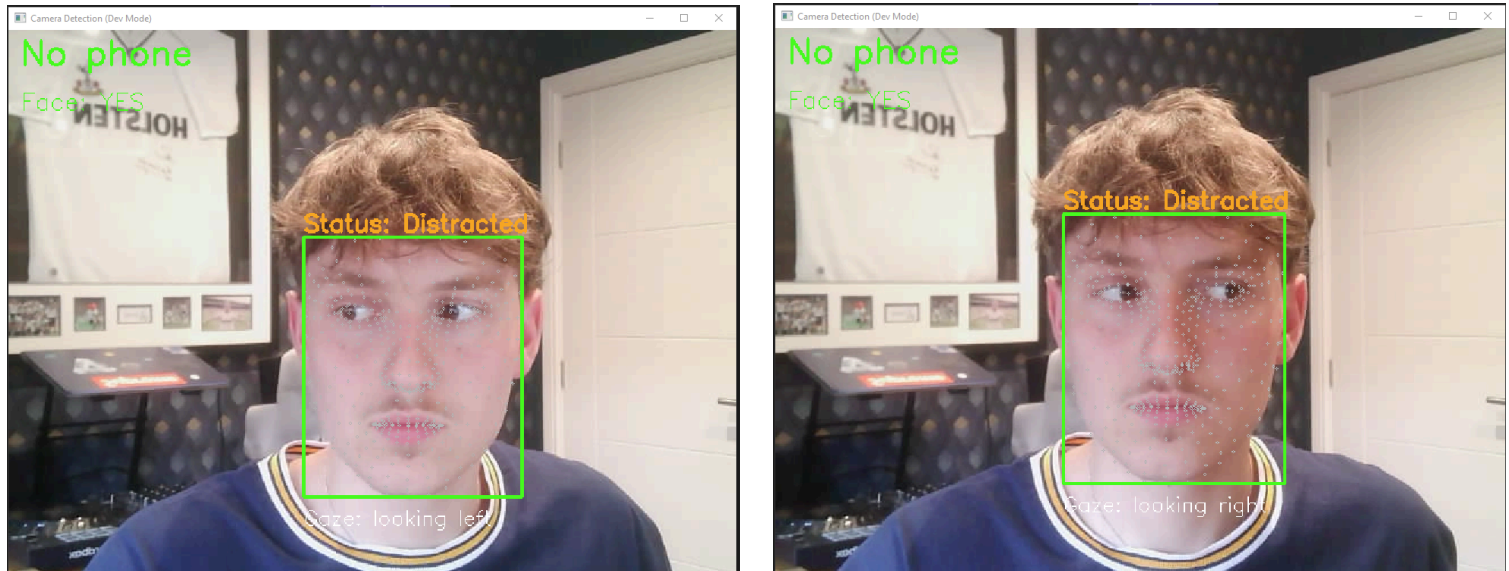
## Technical Implementation

The system processes webcam input and combines face tracking with object detection so that it can determine a users focus state. The footages frames are then captured using [OpenCV](#) and then passed into the detection stage.

Facial landmarks are detected using [MediaPipe](#) face landmarker. These landmarks represent key areas of the face which include the nose, eyes, forehead and chin; these points are used to estimate both your head direction and eye movement.

Head direction is calculated by comparing the position of the nose relative to the span of the face and vertical distance between the forehead and chin. This essentially allows the system to determine whether the user is looking left, right, up or down. While this does give a general indication of where your attention is pointing, it's not always precise just on its own.

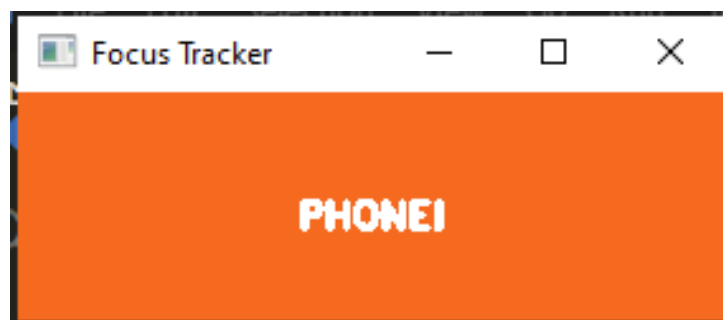
To improve the accuracy of this, **eye direction** is also analysed using the position of the iris with both eyes. By comparing the iris location to the corners of the eyes the system can then detect smaller changes in gaze that the head movement alone would miss



*(Showing eye direction using the position of the iris within each eye)*

These signals are altogether combined to classify whether the user is in one of these three states: **focused**, **distracted** or **away**. The user is only considered to be focused when both the head and eye direction are aligned towards the screen, any other deviation from this results in a distracted state while when there is no face detected the user is classed as “away”. This system only works for laptops or setups with 1 monitor as you would be classed as distracted when looking at a second screen.

Alongside this an object detection model is used to identify mobile phones within a frame. If a phone is detected then the system will immediately classify the user as distracted regardless of any gaze direction. I did this to ensure that phone usage is constantly recognised as a distraction. Usually the user would get 10 seconds before being determined as fully distracted with gaze detection however I made it so that there is no count down and the system immediately flags when there is a phone being detected.

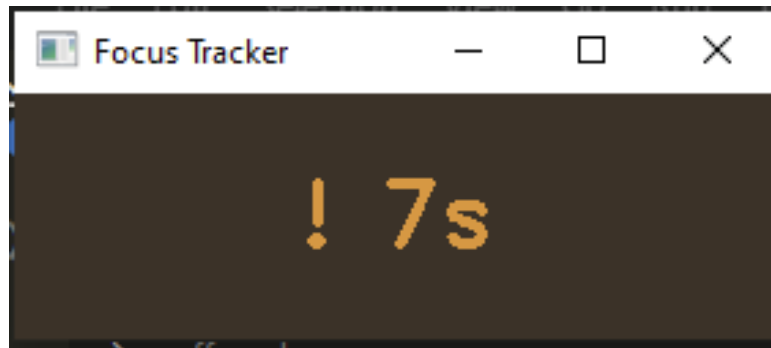


*(Focus tracker widget state when phone is present and detected)*

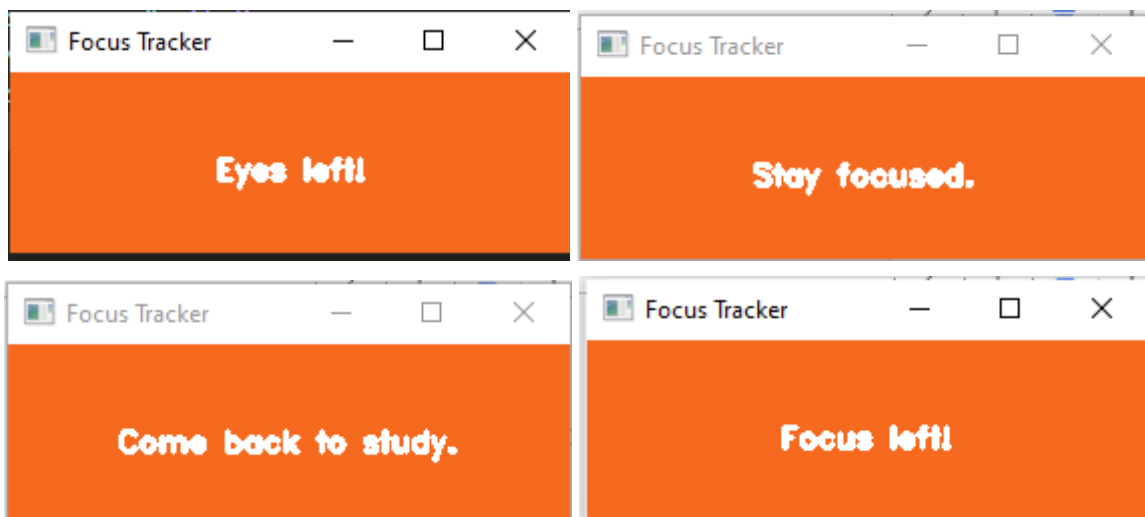
To control the different states in my system I have a central manager component which tracks how long that the user remains in each state and controls when different alert



messages are triggered. To avoid any unnecessary interruptions i added a function so that alerts are only shown after a short period of continuous distraction and are tailored to the type of behaviour thats detected.



*(focus tracker widget showing countdown until alert is triggered)*



*(focus tracker widget showing different alert prompts based on behaviour detected)*

At the end of each session when the user presses ESC a summary data is saved as a JSON file which includes your total study duration, time spent focused, and overall focus score. This allows the user to review their performance and identify patterns over time. The data that is stored in the data/sessions/ file can be copied and then pasted in the "import sessions" page of my fullstack development project where all of your analytics can be shown in charts and graphs to help you view your progress better.

```
data > sessions > {} session_2026-04-27_20-29.json > ...
1  {
2    "date": "2026-04-27",
3    "time": "20:29",
4    "duration_seconds": 194,
5    "focused_seconds": 12,
6    "focus_score": 6.4,
7    "duration_formatted": "03:14",
8    "focused_formatted": "00:12"
9  }
```

# Design Decisions

Several different design decisions were made to ensure that the project i was creating remained practical to use during real study sessions, rather than just becoming a distraction in itself.

One of my initial decisions at the beginning of this project was to create a plant that grows when you are focused but begins to die when you are distracted. After speaking this design decision over with my lecturer and asking for their opinion, we agreed that this implementation would essentially distract the user instead of helping them study, completely destroying the main goal of my project. Talking about my decisions coming from a different view helped me to take a step back and focus on the main task i set out to achieve and to make sure that I wasn't over engineering my project in any way that was unnecessary.

One of the most important decisions i made throughout the project was to separate the camera view from the main interface. The final system runs continuously without constantly displaying the webcam feed with the option to toggle to “developer mode” so you can see the logic behind your distraction state. This again was a decision that i talked over with my lecturer who helped me point out obvious flaws in my project during the development process. If the webcam footage were to stay continuously showing the user their own active camera feed this would definitely reduce the users focus rather than improving it. This change led me to the decision of creating the small “widget” which sits in the corner of your screen, by keeping the interface minimal the system supports concentration instead of interfering with it

The interface itself was designed to be small and unobtrusive being positioned in a way that doesn't block users work. It only becomes more noticeable when an alert is triggered allowing the widget to stay in the background whilst a user works, however when an alert is triggered the widget is set to come to the front of the screen which gives the user a clear indication when they are distracted while still keeping them fully focused on their study session with the tracking going on in the background, unnoticeable.

Another key decision for my design was to use a short delay before triggering alerts when the user is classified as distracted. Instead of reacting instantly, the system waits until distraction has been constant for a brief period. I did this to avoid any unnecessary interruptions and to make sure it accounts for natural behaviour such as briefly looking away from the screen to take a sip of water.

From a development perspective the project was structured into separate modules which each handle a specific responsibility such as detection (  `detection.py` ), alerts (  `alerts.py` ), session management (  `session.py` ) and focus tracking management (  `focus_manager.py` ). This made the system much easier for me to develop, test and modify. I think that this also reflects standard software engineering practice. Overall these design decisions were guided by the idea that the system should integrate naturally into a study environment which would provide useful feedback with trying to not become a source of distraction itself.

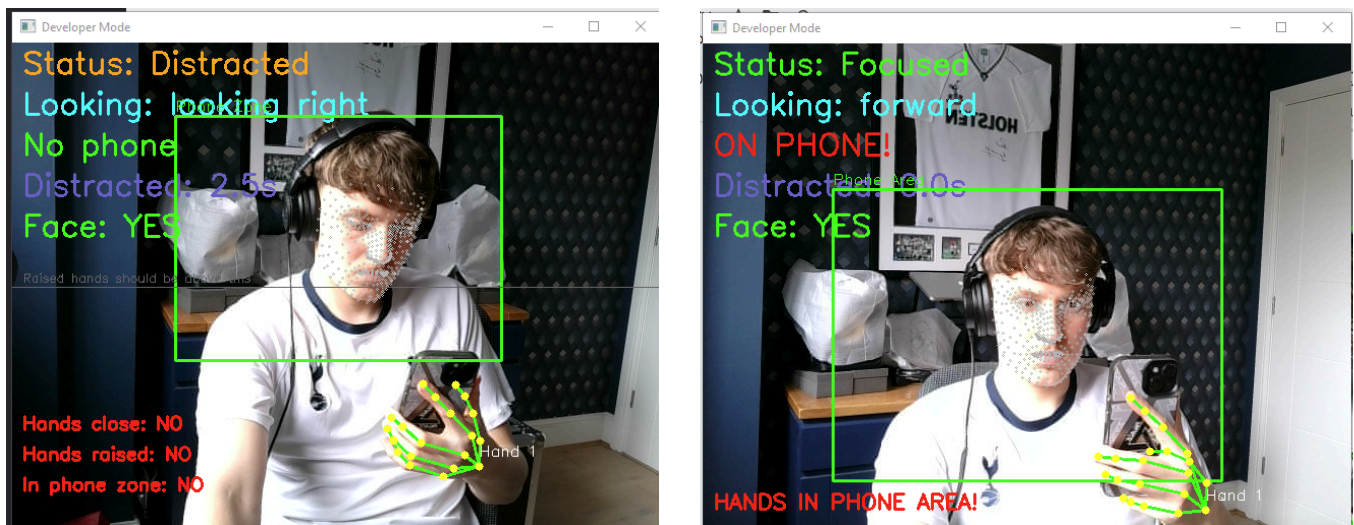
# Development Timeline

The development of this project followed an approach where i added features gradually throughout and improved over time rather than being all built in at once

The project initially began with setting up the development environment and then getting a basic webcam feed working as a starting point. Once i had completed this the next step was to implement face detection to allow the system to recognise whether a user was present or not. This formed the overall foundations to build on for further development.

The next stage involved adding simple head tracking to allow the system to detect when a user was looking away from the screen. This was then later improved by introducing eye tracking using iris positioning, making the system more accurate when trying to detect smaller changes in a user's attention or gaze.

One early attempt at detecting phone usage involved an idea I had to track the hand position within a defined area of the screen. However after testing, this approach proved to be very unreliable as it would pick up different objects inside the box detecting them as phones as it could not consistently distinguish between different types of movement or objects. As a result this method was quickly replaced with the object detection model which significantly improved the system's ability to detect mobile phones.



*(early phone detection using hand tracking)*

In later stages of development my focus switched towards improving the usability of the application. This included things like implementing the alert system refining the user interface and adding the session tracking focus data so it could be recorded over time and added into the fullstack website i was creating.

Overall the system was developed step by step with each part being built on the previous one. This approach made it lots easier to test different specific features and adapt the design when other ideas did not perform as they were expected to



# Ethics and Limitations

Due to having my project use my users webcam footage in real time and also tracking many different facial landmarks there are some ethical considerations that are associated with this project.

One of the main considerations is user privacy. The system overall process all webcam input but does not store any images or video data. The only data that is saved is summary information such as a users session duration and focus score. This ensures that the user's personal data remains private and reduces any risks associated with storing visual data which this project does not do at all.

Another important limitation is that the system can only track visual attention, not a users actual focus. For example, a user may appear to be looking directly at their screen while not actively concentrating on their work. This means that while the system does provide a useful indication of your behaviour it still cant fully determine a user's level of understanding or engagement.

There are also some technical limitations to think about that i found along the way of development. The accuracy of the system can be affected by factors such as lighting conditions, camera quality, and the user's positioning to the camera. In some cases, features such as glasses or unusual angles may reduce the reliability of facial landmark detection. And bad lighting in some areas I've noticed can also reduce reliability of facial landmark detection.

Additionally, as with many computer webcam systems there is potential for bias. Although the models used are trained on large datasets, performance may vary depending on factors such as lighting and appearance. This is an important consideration when evaluating the reliability of the system across different users.

Overall i think that the system provides useful insight into user behaviour however of course its important to recognise its limitations and ensure that it is used as a supportive tool.

## Reflection

Overall i think that the project was successful in achieving what i set out to do. The system is able to detect focus and distraction so that it feels genuinely practical to use in real study sessions. The combination of gaze tracking and phone detection worked the best in my project as it allowed the system to respond to different types of behaviour rather than relying on just one single method which would give my system less reliability.

One of the things im glad i changed in my project was the decision to move away from my initial phone detection idea. The initial approach i implemented using a defined box area and hand tracking did not produce reliable results at all, but recognising this early and switching to a more suitable solution improved the system significantly. This showed me the

importance of testing ideas properly and being willing to change direction when something is not working out.

The modular structure of the project also made development easier. By separating different parts of the system, such as detection, alerts, and session tracking, I was able to work on and test each component individually. This made it easier to fix issues and extend the system over time.

However, there are also areas that definitely could have been improved. The current system uses fixed thresholds to determine gaze direction, which may not work equally well for every user or setup. For example, different camera positions or seating arrangements could affect how accurately attention is detected. Adding some sort of a calibration step could improve this and make the system more adaptable as there are still small limitations like this to my project.

Another limitation is that the system assumes a single screen setup. If a user is working across multiple monitors, looking at a second screen would be classified as distraction. I have tested this multiple times as I usually work on 2 monitors. This highlights how the system is still a simplified model of real world behaviour.

If I were to have any more time and continue developing this project, I would focus on improving the accuracy and personalisation of the system to relate to the user more. This could include better calibration, more advanced behaviour detection, and tighter integration with my full-stack project to provide more detailed feedback and visualisation of focus data over time.

Overall, this project definitely helped me better understand how AI can be applied in a practical context, particularly in combining pre trained models with my own custom logic and ideas to solve a specific problem. It also reinforced the importance of testing, iteration, and making design decisions based on how a system will actually be used.

## Use of Generative AI

In the development of my project I did use generative AI tools such as Claude to help me understand some of the more complex parts of the libraries and documentation that I was working with.

Both [MediaPipe](#) and [OpenCV](#) have really large and detailed documentation which I found very difficult to work through when I was trying to build my ideas at the same time as well as trying to read through large documentation for other modules too. I mainly used AI to help me break down and explain parts of this documentation in a simpler way. For example I asked some questions/prompts such as “which facial landmarks does MediaPipe use for eye tracking and how can I use them to estimate gaze direction?” prompts like these were asked to better understand how I could possibly apply it in my own code

I did not use AI to generate the final implementation of the project. All of the logic used to determine focus, distraction and phone detection was all developed and tested myself i only used ai as a support tool to improve my understanding when i was struggling to find specific and certain information in [Mediapipe](#) and [OpenCV](#) documentation as i found it very complex and time consuming.

I also found that some AI tools were not fully accurate especially when dealing with specific libraries or recently updated APIs. because of this i needed to cross check some explanations with official documentation to fully make sure that what i was implementing was correct

Overall, using generative AI definitely helped me to understand difficult concepts more quickly by breaking documentation down, however I still needed to apply that understanding myself in order to build a working system.

## References

Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.

Google. (2023). *MediaPipe Face Landmarker*. Retrieved from <https://developers.google.com/mediapipe>

Google. (2023). *MediaPipe Object Detection*. Retrieved from <https://developers.google.com/mediapipe>